

Deep Neural Networks via Tensor Calculus

Jeffrey Thompson

December 30, 2025

Abstract

This is my experience with deriving and implementing a fully custom neural network framework. The goal of this write-up is to slowly introduce technical mathematical language and concepts chronologically so that an arbitrary reader can gain some value by reading up to where they can no longer follow. Once reaching this point, an interested reader should have the tools to further explore any concepts presented here and revisit at a later point.

This framework was implemented in Python here: https://github.com/Jaffulee/deep_neural_network

Contents

1	Overview and Motivation	3
1.1	The Goal	3
1.2	The Setup to the Problem	3
2	Gradient Descent	4
2.1	A Brief Dip into Calculus	4
2.2	Gradient Descent Motivation	4
2.3	Gradient Descent in 1 Dimension	5
2.4	A Brief Dip into Multi-Variable Calculus	6
2.5	Gradient Descent in n Dimensions	7
2.6	A Brief Dip into Indices, Matrices and Jacobians	7
3	Revisiting the Goal; Outlining the Idea and Setup	10
3.1	Neural Network with One Layer	10
3.1.1	Neural Networks with One Layer Formulation	14
3.2	How Good is a Single Layer Neural Network?	15
4	Neural Networks with Hidden Layers	16
5	Tensors	19

5.1	Tensor Calculus: the Tensor Jacobian and Gradient	20
6	Neural Networks with Hidden Layers via Tensor Calculus	22
7	Implementation	27

1 Overview and Motivation

The first question is: what do we want to achieve?

1.1 The Goal

The goal is to solve complex non-linear problems (in finite dimensions).

Linear problems are those where scaling the input also scales the output by a similar amount. For example, if an apple costs £1, then 5 apples at that price will cost £5. Linear problems are easy to solve!

Most interesting problems, however, are highly non-linear. For example, given the camera feed from a self-driving car, how can we use that data to steer the car in a sensible direction?

The trouble with non-linear problems is that there may not be a nice way to solve them analytically, and thus we must turn to numeric schemes.

Additionally, in the real world, we often only have a sample of data to work with, and would like to find a way to consistently map the input data to the desired output data.

So more specifically, we would like a way to take samples of inputs, and form a way of non-linearly combining the input data so that we can solve some problem.

1.2 The Setup to the Problem

We are given a dataset $\mathcal{D}$ of m input-output pairs of real-valued vectors $(\hat{\mathbf{x}}_{(i)}, \hat{\mathbf{y}}_{(i)})$;

$$\mathcal{D} = \{(\hat{\mathbf{x}}_{(i)}, \hat{\mathbf{y}}_{(i)}) \mid \hat{\mathbf{x}}_{(i)} \in \mathbb{R}^{n_x}, \hat{\mathbf{y}}_{(i)} \in \mathbb{R}^{n_y}, i = 1, \dots, m\}.$$

Here n_x denotes the number of *input features* (the dimension of each state), and n_y denotes the number of *output features* (the dimension of each target).

Example. In the setup of a self-driving racecar, we might use training data of human drivers on a racetrack, and we could collect input data such as the distance to the side of the road, the angle of the car, the speed and acceleration of the car, etc. in our $\hat{\mathbf{x}}$ vector, with one entry per measurement, and n_x is the number of entries. The outputs collected could then be the decision the driver made such as steering right, braking, accelerating, etc. collected in our $\hat{\mathbf{y}}$ vector, and n_y is the number of decisions. Our number of input-output pairs, m , will be very large in this scenario, since we might be collecting data every few milliseconds, for many laps.

We want to produce a function $\mathbf{y} = \mathbf{y}(\mathbf{x})$ which takes $\mathbf{x}$ as an input such that $\mathbf{y}(\hat{\mathbf{x}}_i) \approx \hat{\mathbf{y}}_i$. We also want to be able to input $\mathbf{x}$ values which are not from $\hat{\mathbf{x}}$, and would like these unknown inputs to produce reasonable results.

Example. In our racecar example, we cannot possibly record enough data to cover every possible input on the racetrack, so we need to be able to reasonably interpolate between the observed values, and be able to make a decision from the output vector similar to what a real driver would do.

Example. Let's say we have a business which helps customers decide whether their fruit is an apple or a pear, based on the colour (average hue), and the roundness (e.g. height - length). $\hat{\mathbf{x}}$ will be a vector of (colour, roundness) and therefore $n_x = 2$, and $\hat{\mathbf{y}}$ will be a vector which indicates either of (apple, pear), hence $n_y = 2$. From this, we have two actions: the fruit is an apple or it is a pear. Let's say, as a business, we have produced a mystical function which takes in (red, round), and outputs (0.99, 1). We can interpret this as our mystical function is 99% sure that the input results in a value of apple.

How do we find such a function $\mathbf{y}$?

2 Gradient Descent

2.1 A Brief Dip into Calculus

If $f : \mathbb{R} \rightarrow \mathbb{R}; x \mapsto f(x)$ is a *function which smoothly takes real numbers to real numbers* (e.g. $f(x) = 2x$), we can consider the *slope of f between two points* to be

$$\frac{\Delta f}{\Delta x} \tag{2.1.1}$$

The delta Δ represents a *small change*; this is the change in f divided by the change in x . If we make the change infinitely small (and repeat this for all values of x), we get *the gradient of f at the point x* , denoted by

$$\frac{df}{dx}$$

It turns out finding the gradient of a lot of functions is relatively simple, by (rigorously) making the small change in Equation 2.1.1 become 0. If we briefly drop rigor¹, and define a composite function $g = g(f(x))$, and since (again non-rigorously) $\Delta f / \Delta f = 1$, we can compute:

$$\frac{\Delta g}{\Delta x} = \frac{\Delta g}{\Delta f} \frac{\Delta f}{\Delta x}$$

Making the change infinitely small (up to rigor), gives us:

Theorem 2.1. *The Chain Rule:*

If $f, g : \mathbb{R} \rightarrow \mathbb{R}$ are suitably smooth and defined, then the composition $g(f(x))$ has the gradient:

$$\frac{dg}{dx} = \frac{dg}{df} \frac{df}{dx}$$

This is an essential Theorem, since it means that we can calculate the gradient of very complex functions as long as we can write that function as a sequence of simpler functions.

2.2 Gradient Descent Motivation

A key fact is that the value of the gradient of a function f at x points in the x -direction of steepest ascent of the function, i.e. where we should move x to increase f by the most. This means if we move x in the direction of the gradient of the function at that point, we can always move in the direction where the function is increasing. Similarly, if we move in the opposite direction, we can move in the direction where the function is decreasing the most.

This is an essential part of how we will construct our $\mathbf{y}$ function from before in Section 1.2; if we have a function that tells us how far away from the correct answer we are, an error or loss function, we can find where the error is smallest by moving x in the direction where the error is decreasing. This can be thought of like rolling a ball down a hill until it reaches the bottom. Unfortunately, there is an issue with this method: if the hill is curvy enough, our ball might get stuck in a ditch that is not the bottom of the hill. We can “fix” this issue by starting with our ball at random parts of the hill, seeing where it rolls down to, and making note of the one that got the lowest.

This concept is the idea behind gradient descent: we want a function E which takes in some number of inputs, and outputs an *error/loss/cost* as a number in which 0 represents the best. Since 0 is the minimum of E , finding the inputs that result in the least error/loss/cost amounts to finding a global minimum. This is very similar to the scenario above; we can try to utilise gradient descent.

¹The presented division works whenever Δf is not 0 near to x , and some more work is needed to make this intuition rigorous, especially around the exchanging of limits and multiplication. Nevertheless, it neatly gives an intuitive foundation for the chain rule.

2.3 Gradient Descent in 1 Dimension

Formulating gradient descent amounts to writing our intuition out accurately.

Theorem 2.2. *Gradient Descent 1D*

Let $E : \mathbb{R} \rightarrow \mathbb{R}^+$ be a non-negative real-valued function which is sufficiently smooth and let $\alpha \in (0, 1]$ be a number between zero and one called the learning rate. Let x_n be a sequence of real numbers defined by the iterative process:

$$x_{n+1} = x_n - \alpha \left. \frac{dE}{dx} \right|_{x=x_n}$$

Where the bar denotes evaluating the gradient at the point x_n . Then, for an appropriately small α ,² x_n approaches a local minimum or a turning point of E as n increases.

By looking at the equation above, one can see that if $\left. \frac{dE}{dx} \right|_{x=x_n}$ is very small (similarly if α is very small), $x_{n+1} \approx x_n$. Points where $\frac{dE}{dx} = 0$ are called turning points, and thus we have justified the final sentence of Theorem 2.2. The problem we face is that turning points are not necessarily minima; we would like to avoid landing on non-minima turning points.

There are various methods to adapt gradient descent to improve the issue of turning points. One approach is to include momentum. Continuing with the ball rolling analogy for gradient descent, we can mimic momentum by calculating the speed at which x itself is moving, and adding that to our formula. This allows for the ball to roll past turning points using the speed it had to reach it.

Momentum can still run into an issue of if the ball perfectly landed on a turning point with no momentum, it can still get stuck. To account for this, we can include an additional force to push the ball every once in a while. This can be thought of as a random gust of wind.

Theorem 2.3. *Gradient Descent with Momentum and Wind (Noise)*

Let $E : \mathbb{R} \rightarrow \mathbb{R}^+$ be a non-negative real-valued function which is sufficiently smooth and let $\alpha, \beta \in (0, 1]$ be numbers between zero and one called the learning rate and the momentum parameter respectively. Let γ be a random number centered around 0, and let $g : \mathbb{N} \rightarrow \mathbb{R}; g : n \mapsto g(n)$ be a real-valued function of the current iteration called the noise parameter. Let x_n be a sequence of real numbers defined by the iterative process:

$$x_{n+1} = x_n - \alpha \left. \frac{dE}{dx} \right|_{x=x_n} - \beta(x_n - x_{n-1}) + g(n)\gamma$$

Where the bar denotes evaluating the gradient at the point x_n . Then, for an appropriate choice of α, β, g , we have that x_n approaches a local minimum of E as n increases.

The parameters α, β, g are referred to as hyperparameters³.

The selection we will make for here $g(n)$ is usually 1 when n is some multiple of a number and 0 otherwise - a burst of noise every fixed number of iterations - and our choice of γ will be usually a normal distribution with mean zero and variance a fixed number.

The learning rate α roughly corresponds to “smaller is more accurate but slower, larger is less accurate but quicker”, and the momentum parameter β can be thought of as the mass of our rolling ball, and 1 represents infinite mass.

²The convergence of this process utilises that the derivative’s magnitude is roughly proportional to how far we are from a local minimum. If α is too large, we may continually overshoot the minimum to where the magnitude of the gradient has not decreased. This would cause us to not converge.

³The word hyper is used here since we are looking at gradient descent from the point of view of a neural network, where the gradient descent parameters are seen as pre-chosen. Choosing appropriate hyperparameters can be difficult.

The great news is that gradient descent in Theorem 2.3 only really depends on the gradient at a point, and storing some extra previous or random numbers. This makes it quite simple to implement in code, as long as you have a sensible way of getting the gradient.

2.4 A Brief Dip into Multi-Variable Calculus

We care about vectors due to our setup of the problem in Section 1.2. What if a function depends on a vector, i.e. what if it has many input variables $f = f(x^1, \dots, x^n)$? Note that the superscript is just indexing the argument component⁴, not taking a number to a power. We want a way of seeing how f changes with respect to its input variables. For convenience, let us write $f = f(\mathbf{x})$, where $\mathbf{x} = (x^1, \dots, x^n)^T$. Note T denotes *transposing* so that $\mathbf{x}$ is a column vector.

We can reuse our notion of a gradient for a function with one variable from Equation 2.1.1 by fixing all but one of the x^i s, and computing the gradient of the remaining x^i as before. To distinguish this process from the one-variable process, we call this a *partial derivative of f with respect to x^i* and denote it with “del”, a curly d:

$$\frac{\partial f}{\partial x^i}$$

Now we can reuse our notion of a gradient by calculating the partial derivative with respect to each x^i in $\mathbf{x}$ separately.

Definition 2.4. The gradient of a function with multiple variables.

Define the gradient of $f = f(\mathbf{x})$, denoted $\nabla_{\mathbf{x}} f$ to be:

$$\nabla_{\mathbf{x}} f = \left(\frac{\partial f}{\partial x^1}, \dots, \frac{\partial f}{\partial x^n} \right)^T$$

Importantly, the gradient still points in the direction of steepest ascent of f with respect to the input variables $\mathbf{x}$. Thus, we can extend gradient descent to a function of multiple variables by conducting gradient descent on each component.

Before doing this, we also need to introduce the concept of a gradient when we have nested functions. Let’s say that each x^i is itself a function of m variables, i.e. $\mathbf{x} = \mathbf{x}(\mathbf{w})$, where $\mathbf{w} = (w^1, \dots, w^m)^T$:

We would like to know how f changes when one of the w^i changes.

$$\mathbf{x} = \mathbf{x}(\mathbf{w}) = \begin{bmatrix} x^1(\mathbf{w}) \\ x^2(\mathbf{w}) \\ \vdots \\ x^n(\mathbf{w}) \end{bmatrix} = \begin{bmatrix} x^1(w^1, \dots, w^m) \\ x^2(w^1, \dots, w^m) \\ \vdots \\ x^n(w^1, \dots, w^m) \end{bmatrix}$$

Definition 2.5. Chain Rule for Multi-Variate Functions

Let $f = f(\mathbf{x})$ be a function of n variables, where each variable is a function of m variables $\mathbf{w}$, i.e. $\mathbf{x} = \mathbf{x}(\mathbf{w})$. We can write $f = f(\mathbf{x}(\mathbf{w}))$. Then we define the partial derivative of f with respect to w^i , denoted $\frac{\partial f}{\partial w^i}$, is the following:

$$\frac{\partial f}{\partial w^i} = \frac{\partial f}{\partial x^1} \frac{\partial x^1}{\partial w^i} + \dots + \frac{\partial f}{\partial x^n} \frac{\partial x^n}{\partial w^i}$$

Which can be thought of like “looking at how f changes as you nudge each variable of f by a little bit in one direction.” Note that this is not quite the previous chain rule, and justifying this definition takes a little more work for the interested reader.

⁴Superscripts for indexing vectors is a convention utilised in differential geometry which helps with book-keeping later.

2.5 Gradient Descent in n Dimensions

Extending gradient descent to n dimensions, it is mostly an exercise in notation. If α is a real number, and $\mathbf{x} = (x^1, \dots, x^n)^T$ is a vector, we define $\alpha\mathbf{x}$ to be component-wise multiplication by α ,

$$\alpha\mathbf{x} = (\alpha x^1, \dots, \alpha x^n)^T$$

If $\mathbf{y} = (y^1, \dots, y^n)^T$ is another vector of the same length, then we define $\mathbf{x} + \mathbf{y}$ to be relative component-wise addition,

$$\mathbf{x} + \mathbf{y} = (x^1 + y^1, \dots, x^n + y^n)^T$$

Now, our notation allows us to write a familiar gradient descent algorithm.

Theorem 2.6. *Gradient Descent in N Dimensions with Momentum and Wind (Noise)*

Let $E : \mathbb{R}^N \rightarrow \mathbb{R}^+$ be a non-negative real-valued function of N variables which is sufficiently smooth and let $\alpha, \beta \in (0, 1]$ be numbers between zero and one called the learning rate and the momentum parameter respectively. Let $\boldsymbol{\gamma}$ be a vector of N random numbers centered around 0, and let $g : \mathbb{N} \rightarrow \mathbb{R}$; $g : n \mapsto g(n)$ be a real-valued function of the current iteration called the noise parameter. Let $\mathbf{x}_n$ be a sequence of length N vectors defined by the iterative process:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \alpha \nabla_{\mathbf{x}} E|_{\mathbf{x}=\mathbf{x}_n} - \beta(\mathbf{x}_n - \mathbf{x}_{n-1}) + g(n)\boldsymbol{\gamma}$$

Where the bar denotes evaluating the gradient at each point $\mathbf{x}_n$ respectively. Then, for an appropriate choice of α, β, g , we have that $\mathbf{x}_n$ approaches a local minimum of E as n increases.

The parameters α, β, g are referred to as hyperparameters.

2.6 A Brief Dip into Indices, Matrices and Jacobians

A matrix is a two-dimensional array of numbers.

$$A = \begin{bmatrix} A^1_1 & A^1_2 & \cdots & A^1_m \\ A^2_1 & A^2_2 & \cdots & A^2_m \\ \vdots & \vdots & \ddots & \vdots \\ A^n_1 & A^n_2 & \cdots & A^n_m \end{bmatrix}$$

We say A is an $(n \times m)$ matrix if A has n rows and m columns, and we can refer to each of the individual values with index notation A^i_j , where i ranges over the rows (vertically) and j ranges over the columns (horizontally). Note that we can consider a length n (column) vector as being a $(n \times 1)$ matrix. Moreover, it is sometimes useful to consider a matrix as a group of m (column) vectors, e.g. if $\mathbf{a}_{(1)}, \dots, \mathbf{a}_{(m)}$ are column vectors defined by $\mathbf{a}_{(j)}^i = A^i_j$, we can write:

$$A = \begin{bmatrix} A^1_1 & A^1_2 & \cdots & A^1_m \\ A^2_1 & A^2_2 & \cdots & A^2_m \\ \vdots & \vdots & \ddots & \vdots \\ A^n_1 & A^n_2 & \cdots & A^n_m \end{bmatrix} = \begin{bmatrix} \uparrow & \uparrow & \cdots & \uparrow \\ \mathbf{a}_{(1)} & \mathbf{a}_{(2)} & \cdots & \mathbf{a}_{(m)} \\ \downarrow & \downarrow & \cdots & \downarrow \end{bmatrix}$$

Given an $(n \times m)$ matrix A and an $(m \times k)$ matrix B , we can define an $(n \times k)$ matrix AB by

$$(AB)^i_j = A^i_1 B^1_j + A^i_2 B^2_j + \cdots + A^i_m B^m_j$$

For convenience, we will use a shorthand called Einstein Summation Notation, where we sum over repeated indices:

$$(AB)^i{}_j = A^i{}_1 B^1{}_j + A^i{}_2 B^2{}_j + \cdots + A^i{}_m B^m{}_j = A^i{}_\rho B^\rho{}_j$$

Similar to with vectors in the previous section, if α is a real number, and A is a matrix, we can define αA to be component-wise multiplication by α , and if B is another matrix with the same dimensions of A , we can define $A + B$ to be relative component-wise addition.

Up until now, we have only considered functions with many inputs, and not functions with many outputs. Let $\mathbf{y} : \mathbb{R}^m \rightarrow \mathbb{R}^n$; $\mathbf{x} \mapsto \mathbf{y}(\mathbf{x})$ be an n -dimensional vector-valued function of m variables.

$$\mathbf{y}(\mathbf{x}) = \begin{bmatrix} y^1(\mathbf{x}) \\ y^2(\mathbf{x}) \\ \vdots \\ y^m(\mathbf{x}) \end{bmatrix}$$

Since we care about how functions change, it is useful to define an object called the Jacobian, which can be thought of as a generalisation of the gradient.

Definition 2.7. The Jacobian Matrix

Let $\mathbf{y} : \mathbb{R}^n \rightarrow \mathbb{R}^m$; $\mathbf{x} \mapsto \mathbf{y}(\mathbf{x})$ be a sufficiently smooth m -dimensional vector-valued function of n variables. Define the *Jacobian matrix* $D_{\mathbf{x}}\mathbf{y}$ by

$$D_{\mathbf{x}}(\mathbf{y})^i{}_j = \frac{\partial y^i}{\partial x^j}$$

In matrix form,

$$D_{\mathbf{x}}\mathbf{y} = \begin{bmatrix} \frac{\partial y^1}{\partial x^1} & \frac{\partial y^1}{\partial x^2} & \cdots & \frac{\partial y^1}{\partial x^n} \\ \frac{\partial y^2}{\partial x^1} & \frac{\partial y^2}{\partial x^2} & \cdots & \frac{\partial y^2}{\partial x^n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y^m}{\partial x^1} & \frac{\partial y^m}{\partial x^2} & \cdots & \frac{\partial y^m}{\partial x^n} \end{bmatrix} = \begin{bmatrix} \leftarrow & \nabla_{\mathbf{x}} y^1 & \rightarrow \\ \leftarrow & \nabla_{\mathbf{x}} y^2 & \rightarrow \\ & \vdots & \\ \leftarrow & \nabla_{\mathbf{x}} y^m & \rightarrow \end{bmatrix}$$

Notice that this means we can rewrite the gradient of a function $f = f(\mathbf{x})$ of multiple variables from Definition 2.4 as

Proposition 2.8. The gradient of a function in terms of the Jacobian matrix.

The gradient of $f = f(\mathbf{x})$, $\nabla_{\mathbf{x}} f$ can be written in terms of the Jacobian of f as:

$$\nabla_{\mathbf{x}} f = (D_{\mathbf{x}} f)^T$$

The transpose T in the above is included since the gradient is a column vector.

Due to the definition, this matrix encodes how each of the y^i functions change as $\mathbf{x}$ changes, making it useful for our goal.

Where this Jacobian becomes especially useful is by looking more closely at our definition of the multi-variate chain rule from Definition 2.5 for a function f of n variables $\mathbf{y}$, and where $\mathbf{y}$ is a function of m variables $\mathbf{x}$.

We can first rewrite our total derivative using Einstein Summation Convention (sum over repeated indices) as:

$$\frac{\partial f}{\partial x^j} = \frac{\partial f}{\partial y^\rho} \frac{\partial y^\rho}{\partial x^j}$$

Where now we can associate each term in the multiplication as multiplication of matrices via:

$$\frac{\partial f}{\partial x^j} = \frac{\partial f}{\partial y^\rho} \frac{\partial y^\rho}{\partial x^j} = \begin{bmatrix} \frac{\partial f}{\partial y^1} & \frac{\partial f}{\partial y^2} & \cdots & \frac{\partial f}{\partial y^m} \end{bmatrix} \begin{bmatrix} \frac{\partial y^1}{\partial x^j} \\ \frac{\partial y^2}{\partial x^j} \\ \frac{\partial y^3}{\partial x^j} \\ \vdots \\ \frac{\partial y^m}{\partial x^j} \end{bmatrix}$$

We can continue to utilise matrix notation by expanding each j into columns of a matrix:

$$\begin{bmatrix} \frac{\partial f}{\partial x^1} & \cdots & \frac{\partial f}{\partial x^n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial y^1} & \frac{\partial f}{\partial y^2} & \cdots & \frac{\partial f}{\partial y^m} \end{bmatrix} \begin{bmatrix} \frac{\partial y^1}{\partial x^1} & \frac{\partial y^1}{\partial x^2} & \cdots & \frac{\partial y^1}{\partial x^n} \\ \frac{\partial y^2}{\partial x^1} & \frac{\partial y^2}{\partial x^2} & \cdots & \frac{\partial y^2}{\partial x^n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y^m}{\partial x^1} & \frac{\partial y^m}{\partial x^2} & \cdots & \frac{\partial y^m}{\partial x^n} \end{bmatrix}$$

Using the definition of the Jacobian in Definition 2.7, we can rewrite this neatly as:

$$D_{\mathbf{x}}f = D_{\mathbf{y}}f D_{\mathbf{x}}\mathbf{y}$$

This encodes a similar notion to the chain rule in Theorem 2.1. We can extend this similarly when f itself is a function with many outputs, getting the following essential result.

Theorem 2.9. *Generalised Chain Rule for Multi-variate functions*

Let $\mathbf{z} = \mathbf{z}(\mathbf{y})$ be an n -variable function of m variables $\mathbf{y}$, and let $\mathbf{y} = \mathbf{y}(\mathbf{x})$ be an m -variable function of n variables $\mathbf{x}$. Then

$$D_{\mathbf{x}}\mathbf{z} = D_{\mathbf{y}}\mathbf{z} D_{\mathbf{x}}\mathbf{y}$$

Up until this point, this has felt quite unrelated to our goal. Except, we can relate it directly back to our goal and setup in Section 1.2 neatly.

3 Revisiting the Goal; Outlining the Idea and Setup

As a reminder, the setup to our problem is as follows, using the notation in the previous sections.

The goal is to solve complex non-linear problems (in finite dimensions).

We are given a dataset $\mathcal{D}$ of m input-output pairs of real-valued vectors $(\hat{\mathbf{x}}_{(i)}, \hat{\mathbf{y}}_{(i)})$;

$$\mathcal{D} = \{(\hat{\mathbf{x}}_{(i)}, \hat{\mathbf{y}}_{(i)}) \mid \hat{\mathbf{x}}_{(i)} \in \mathbb{R}^{n_x}, \hat{\mathbf{y}}_{(i)} \in \mathbb{R}^{n_y}, i = 1, \dots, m\}.$$

Here n_x denotes the *number of input features* (the dimension of each state), and n_y denotes the *number of output features* (the dimension of each target).

We want to produce a function $\mathbf{y} = \mathbf{y}(\mathbf{x})$ which takes $\mathbf{x}$ as an input such that $\mathbf{y}(\hat{\mathbf{x}}_i) \approx \hat{\mathbf{y}}_i$. We also want to be able to input $\mathbf{x}$ values which are not from $\hat{\mathbf{x}}$, and would like these unknown inputs to produce reasonable results.

To find such a $\mathbf{y}$, we would like to utilise gradient descent. We also want to be able to simply compute the gradient descent algorithm, meaning we would like it to be written as a sequence of iteratively applied – potentially multi-variate – functions so that we can apply the generalised chain rule in Theorem 2.9.

Specifically, we will approach this task by finding a function $\mathbf{y}$ with the following framework:

1. Choose $\mathbf{y}$ to be a function of multiple variables, $\mathbf{x}$ and $\mathbf{w}$.
2. For a choice of $\mathbf{w}$, check what $\mathbf{y}(\hat{\mathbf{x}}_{(j)})$ is.
3. Calculate a choice of error E as a function of $\mathbf{y}$ by comparing $\mathbf{y}(\hat{\mathbf{x}}_{(j)})$ with the desired output $\hat{\mathbf{y}}_{(j)}$.
4. Consider E as a function of $\mathbf{w}$, and vary $\mathbf{w}$.
5. Minimise the overall error by using gradient descent on E with respect to the variables $\mathbf{w}$.

We can see the generalised chain rule Theorem 2.9 appear explicitly here via the following, also recalling Proposition 2.8, and writing $E = E(\mathbf{y}(\mathbf{w}; \mathbf{x}))$, where the semicolon is denoting that we are fixing $\mathbf{x}$ to consider our variables of $\mathbf{y}$ to be only $\mathbf{w}$:

$$\nabla_{\mathbf{w}} E = (D_{\mathbf{w}} E)^T = (D_{\mathbf{y}} E D_{\mathbf{w}} \mathbf{y})^T$$

Additionally, we want the following properties to guide us with how to achieve the goal:

1. E and $\mathbf{y}$ need to be chosen so that we can calculate their Jacobian matrices simply.
2. $\mathbf{y}$ needs to be a non-linear function to allow for it to solve interesting problems.

3.1 Neural Network with One Layer

Where do we get the variables $\mathbf{w}$ from? Let's temporarily drop the requirement for non-linearity, and define $\mathbf{y}$ component-wise to be a linear combination of each x^j :

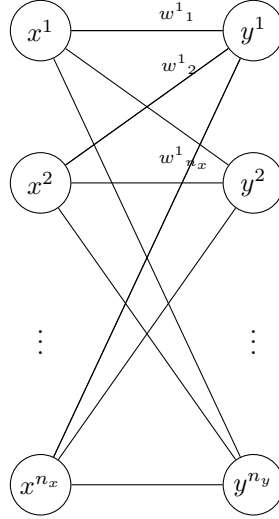
$$y^i = w^i_j x^j + b^i$$

Where we are summing over repeated indices in $w^i_j x^j$. We can write this neatly as a matrix equation by using the matrix W for the entries w^i_j and using the (column) vector $\mathbf{b}$ for the entries b^i :

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}$$

The w are often called *weights* and the b are often called *bias*.

A handy way to visualise this is via the following graph, where we can imagine each y to be made up of multiplying the edge w with the connected x , along with adding b to each y .



If we define $\mathbf{w}$ as a column vector with all of the entries of W in one column, we are interested in the two quantities: $D_{\mathbf{w}}\mathbf{y}$ and $D_{\mathbf{b}}\mathbf{y}$. These will allow us to compute the gradient of the error later with respect to these inputs.

$$\mathbf{w} = \begin{bmatrix} w^1_1 \\ w^1_2 \\ \vdots \\ w^1_m \\ w^2_1 \\ \vdots \\ w^n_m \end{bmatrix}$$

The benefit of using a linear combination is that it is “simple” to find the Jacobian matrix. First we introduce the *Kronecker delta* δ^i_j to be 1 when $i = j$ and 0 otherwise. This is essential because $\frac{\partial x^i}{\partial x^j} = \delta^i_j$ for independent variables. Note also that $\delta^i_\rho x^\rho = x^i$, where we are summing over repeated variables. Then we calculate (slightly abusing matrix notation and using shorthand):

$$\begin{aligned} (D_{\mathbf{w}}\mathbf{y}) &= \frac{\partial y^i}{\partial w^j_k} \\ &= \partial_{w^j_k}(y^i) && \text{(shorthand for partial derivative)} \\ &= \partial_{w^j_k}(w^i_\rho x^\rho + b^i) && \text{(substituting } y \text{ definition)} \\ &= x^\rho \partial_{w^j_k}(w^i_\rho) && \text{(linearity of partial derivative)} \\ &= x^\rho \delta^i_j \delta^k_\rho && \text{(independence of variables)} \\ &= x^k \delta^i_j && \text{(summing over Kronecker delta)} \end{aligned}$$

$$\begin{aligned}
(D_{\mathbf{b}}\mathbf{y}) &= \frac{\partial y^i}{\partial b^j} \\
&= \partial_{b^j}(y^i) && \text{(shorthand for partial derivative)} \\
&= \partial_{b^j}(w^i_{\rho}x^{\rho} + b^i) && \text{(substituting } y \text{ definition)} \\
&= \partial_{b^j}(b^i) && \text{(linearity of partial derivative)} \\
&= \delta^i_j
\end{aligned}$$

Again, we consider $D_{\mathbf{w}}\mathbf{y}$ to be a matrix by considering $\mathbf{w}$ as a vector.

It will also be useful to compute the next quantity for later.

$$\begin{aligned}
(D_{\mathbf{x}}\mathbf{y}) &= \frac{\partial y^i}{\partial x^j} \\
&= \partial_{x^j}(y^i) && \text{(shorthand for partial derivative)} \\
&= \partial_{x^j}(w^i_{\rho}x^{\rho}) && \text{(substituting } y \text{ definition)} \\
&= w^i_{\rho}\partial_{x^j}(x^{\rho}) && \text{(linearity of partial derivative)} \\
&= w^i_{\rho}\delta^{\rho}_j && \text{(independence of variables)} \\
&= w^i_j && \text{(summing over Kronecker delta)}
\end{aligned}$$

Hence we have found the following.

Proposition 3.1. *Define $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ for a matrix W and vectors $\mathbf{x}$ and $\mathbf{b}$. Let δ denote the Kronecker delta δ^i_j to be 1 when $i = j$ and 0 otherwise. Consider W as a vector via the following construction.*

$$\mathbf{w} = \begin{bmatrix} w^1_1 \\ w^1_2 \\ \vdots \\ w^1_m \\ w^2_1 \\ \vdots \\ w^n_m \end{bmatrix}$$

Then we have the following:

1. $(D_{\mathbf{w}}\mathbf{y})^i_{jk} = x^k\delta^i_j$
2. $(D_{\mathbf{b}}\mathbf{y})^i_j = \delta^i_j$
3. $(D_{\mathbf{x}}\mathbf{y})^i_j = w^i_j$

Where we consider the former as a matrix by converting the lower (jk) indices into a vector as we did with W itself.

Let's also choose an $E = E(\mathbf{y})$ which we can compute the Jacobian matrix of "simply". Let $\|\cdot\|$ denote the Euclidean norm and define E as:

$$E(\mathbf{y}; \hat{\mathbf{y}}_{(i)}) = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}_{(i)}\|^2 = \frac{1}{2}(\mathbf{y} - \hat{\mathbf{y}}_{(i)})^T(\mathbf{y} - \hat{\mathbf{y}}_{(i)})$$

Then consider

$$\begin{aligned}
(D_{\mathbf{y}}E)_j &= \frac{\partial E}{\partial y^j} \\
&= \frac{1}{2} \partial_{y^j} ((y^j)^2 - 2y^j \hat{y}^j - (\hat{y}^j)^2) \quad (\text{no sum over } j) \\
&= \frac{1}{2} (2y^j - 2\hat{y}^j) \\
&= y^j - \hat{y}^j
\end{aligned}$$

Hence, carefully keeping track of indices, we find the following.

Proposition 3.2. *Let $\|\cdot\|$ denote the Euclidean norm and define E as:*

$$E(\mathbf{y}; \hat{\mathbf{y}}_{(i)}) = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}_{(i)}\|^2$$

Then

$$D_{\mathbf{y}}E(\mathbf{y}; \hat{\mathbf{y}}) = (\mathbf{y} - \hat{\mathbf{y}})^T$$

Therefore, we can finally compute:

$$\nabla_{\mathbf{w}}E = (D_{\mathbf{w}}E)^T = (D_{\mathbf{y}}ED_{\mathbf{w}}\mathbf{y})^T$$

Hence, we can substitute the above formulae to conduct gradient descent Theorem 2.6 and minimise E with respect to $\mathbf{w}$ and $\mathbf{b}$. Unfortunately while this would work, this is completely linear, and therefore will be able to find linear solutions.

How do we introduce non-linearity? Moreover, how do we ensure that introducing non-linearity still enables us to compute the Jacobian simply?

The simplest way to introduce some non-linearity is to add a non-linear function between y and E .

To account for this, let's introduce a (non-linear) function σ which takes in a vector and outputs a vector of the same length, and also do some relabeling and book-keeping:

$$\begin{aligned}
\mathbf{z} &= W\mathbf{x} + \mathbf{b} \\
\mathbf{y} &= \sigma(\mathbf{z})
\end{aligned}$$

Moreover, if σ is defined such that it is a component-wise application of a function σ , then

$$(D_{\mathbf{z}}\sigma)^i_j = \frac{d\sigma}{dz^j}$$

An important choice of σ (that we extend component-wise to σ) which achieves our desire is the *sigmoid* function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

This is because the sigmoid function is non-linear, and we can write the gradient of σ in terms of

σ itself:

$$\begin{aligned}
\frac{d\sigma}{dx}(x) &= e^{-x} \frac{1}{(1 + e^{-x})^2} \\
&= \sigma(x) \frac{e^{-x}}{1 + e^{-x}} \\
&= \sigma(x) \frac{(1 + e^{-x}) - 1}{1 + e^{-x}} \\
&= \sigma(x) \left(1 - \frac{1}{1 + e^{-x}} \right) \\
&= \sigma(x)(1 - \sigma(x))
\end{aligned}$$

Hence, we find that

Proposition 3.3. *If σ is constructed as the component-wise extension of the sigmoid function σ ,*

$$(D_z \sigma)_j^i = \frac{d\sigma}{dz^j} = \sigma(z^j)(1 - \sigma(z^j))$$

3.1.1 Neural Networks with One Layer Formulation

So we have the setup

$$\mathbf{z} = W\mathbf{x} + \mathbf{b} \tag{3.1.1}$$

$$\mathbf{y} = \sigma(\mathbf{z}) \tag{3.1.2}$$

$$E = E(\mathbf{y}; \hat{\mathbf{y}}) = E(\sigma(\mathbf{z}); \hat{\mathbf{y}}) \tag{3.1.3}$$

The components of W are called *weights* and the components of $\mathbf{b}$ are called *bias*.

We can now combine the results from Theorem 2.9 and Propositions 3.1, 3.3, and 3.2⁵. to get the main result of this section.

Theorem 3.4. *Gradient descent formula for a simple neural network.*

Given the setup above, we have that

$$\nabla_w E = (D_\sigma E D_z \sigma D_w \mathbf{z})^T$$

$$\nabla_b E = (D_\sigma E D_z \sigma D_b \mathbf{z})^T$$

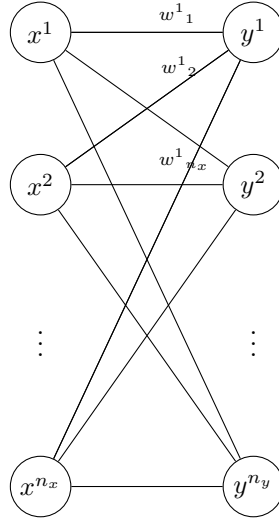
These can be fed into the gradient descent algorithm Theorem 2.6.

The main thing that makes the above possible to interpret as an algorithm, is that the results of each Jacobian above can be written in terms of the variables $\mathbf{z}$, $\mathbf{y}$, and $\mathbf{x}$. Hence, given a fixed $\mathbf{x} = \hat{\mathbf{x}}$ and some numbers (e.g. initially randomly chosen) weights W and bias $\mathbf{b}$, we can find how our error function E changes with respect to W and $\mathbf{b}$ with only computing Equations 3.1.1, 3.1.2, and 3.1.3. Computing Equations 3.1.1, 3.1.2, and 3.1.3 involves feeding numbers through each in order, this is often called *forward propagation*, or a *forward pass*. Computing the gradients in Theorem 3.4 is then often called *backward propagation*.

Due to our knowledge of gradient descent, we know that this process will find a (local) minima of E when viewed as a function of the weights and bias, W and $\mathbf{b}$.

This whole process is referred to as a *neural network with one layer* (the “one layer” can be thought of as being one set of lines between $\mathbf{x}$ and $\mathbf{y}$):

⁵The latter two are based on our choice of σ and E ; we have chosen σ to be the sigmoid function, and E to be the square error for consistency



A limitation which needs to be addressed is with our choice of error function in Proposition 3.2. This function by definition only computes the error for one $\hat{\mathbf{y}}$. Our setup in Section 3 requires a collection of such $\hat{\mathbf{y}}_{(i)}$. One approach is to calculate one step of gradient descent for each pair $(\hat{\mathbf{x}}_{(i)}, \hat{\mathbf{y}}_{(i)})$ respectively, and take the average of the new $\mathbf{x}$ as the “actual” step. We will instead take another approach which also removes this issue once tensors have been introduced.

3.2 How Good is a Single Layer Neural Network?

How well does this neural network achieve our goal in Section 3? While it is true that this will minimise the error, this might still be useless for complex problems if we have not encoded enough non-linearity. Considering σ as the component-wise sigmoid function, let’s unravel what $\mathbf{y}$ is:

$$\begin{aligned}
 (\mathbf{y})^i &= (\sigma(\mathbf{z}))^i \\
 &= (\sigma(W\mathbf{x} + \mathbf{b}))^i \\
 &= \sigma(w^i_j x^j + b^i) \\
 &= \frac{1}{1 + e^{-(w^i_j x^j + b^i)}} \\
 &= \frac{1}{1 + e^{-w^i_1 x^1} e^{-w^i_2 x^2} \dots e^{-b^i}}
 \end{aligned}$$

This means that our non-linearity is restricted to specifically the form of (the quotient of) some multiplications. This is not “very” non-linear, and results in this neural network not being powerful enough to solve complicated non-linear problems.

Also notice that we can now understand why the components of $\mathbf{b}$ are called bias: the denominator has the term e^{-b^i} , which pushes the entire result towards 0 or 1 when b^i becomes very negative or positive respectively, introducing an overall *bias* towards 0 or 1.

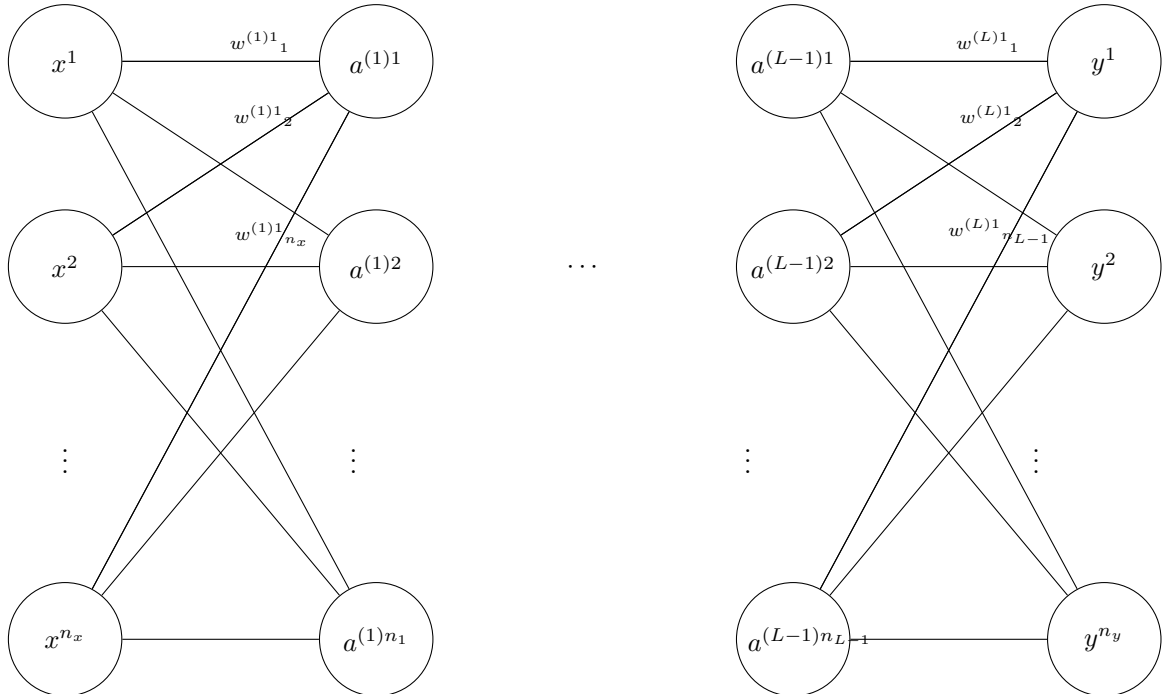
4 Neural Networks with Hidden Layers

As discussed in the previous section, neural networks with one layer suffer from a lack of sufficient non-linearity.

To keep aligned with our goal of simplicity of computation in Section 3, we can reuse the work that led to Theorem 3.4 by inserting more iterations of applying weights and then a non-linear function σ . We call these intermediate layers *hidden layers*. Let L be a number more than 0 called *the number of layers*. Define a sequence of matrices $W^{(i)}$, vectors $\mathbf{z}^{(i)}$, $\mathbf{a}^{(i)}$, and $\mathbf{b}^{(i)}$, and *activation functions* $\sigma^{(i)}$ which take vectors to vectors of the same dimension, for $i = 1, \dots, L$ by:

$$\begin{aligned} \mathbf{a}^{(0)} &= \mathbf{x} \\ \mathbf{z}^{(i)} &= W^{(i)} \mathbf{a}^{(i-1)} + \mathbf{b}^{(i)} \\ \mathbf{a}^{(i)} &= \sigma^{(i)}(\mathbf{z}^{(i)}) \\ \mathbf{y} &= \mathbf{a}^{(L)} \\ E &= E(\mathbf{y}; \hat{\mathbf{y}}) \end{aligned} \tag{4.0.1}$$

The components of $W^{(i)}$ and $\mathbf{b}^{(i)}$ are called the *weights and bias for layer i* respectively. Note that we are free to choose each $\mathbf{a}^{(i)}$ to be any dimension (size) we would like whenever i is not 0 or L , where it must have the same dimension as $\mathbf{x}$ and $\mathbf{y}$ respectively. Moreover, given a choice of dimensions of each $\mathbf{a}^{(i)}$ as n_i , the dimensions of each $W^{(i)}$ are determined to be $(n_i \times n_{i-1})$ respectively, and $\mathbf{b}^{(i)}$ to be a vector of length n_i . Note we interpret $n_0 = n_x$ and $n_L = n_y$. We can visualise this again in graph form as the following:



Now we have many parameters that we wish to vary to minimise our error function E . We can repeatedly apply the Chain Rule Theorem 2.9 to compute each quantity we desire by considering E as a function of the weights and bias, unraveling the sequence in Sequence 4.0.1. Moreover, we can collapse each matrix $W^{(i)}$ into a vector $\mathbf{w}^{(i)}$ as in Proposition 3.1.

For example, temporarily suppressing $\hat{\mathbf{y}}$ for clarity, we may unravel Sequence 4.0.1 until we find $\mathbf{w}$ or $\mathbf{b}$:

$$\begin{aligned}
E &= E(\mathbf{y}) = E\left(\mathbf{a}^{(L)}\right) \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\right)\right) \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\mathbf{w}^{(L)}\right)\right)\right) && \text{(found a } \mathbf{w} \text{)} \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\mathbf{b}^{(L)}\right)\right)\right) && \text{(found a } \mathbf{b} \text{)} \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\mathbf{a}^{(L-1)}\right)\right)\right) \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\boldsymbol{\sigma}^{(L-1)}\left(\mathbf{z}^{(L-1)}\right)\right)\right)\right) \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\boldsymbol{\sigma}^{(L-1)}\left(\mathbf{z}^{(L-1)}\left(\mathbf{w}^{(L-1)}\right)\right)\right)\right)\right) && \text{(found a } \mathbf{w} \text{)} \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\boldsymbol{\sigma}^{(L-1)}\left(\mathbf{z}^{(L-1)}\left(\mathbf{b}^{(L-1)}\right)\right)\right)\right)\right) && \text{(found a } \mathbf{b} \text{)} \\
E &= E\left(\boldsymbol{\sigma}^{(L)}\left(\mathbf{z}^{(L)}\left(\boldsymbol{\sigma}^{(L-1)}\left(\mathbf{z}^{(L-1)}\left(\mathbf{a}^{(L-2)}\right)\right)\right)\right)\right) \\
&\vdots
\end{aligned}$$

Due to this unraveling process, it is also helpful to compute the gradients “backwards” via repeatedly applying the Chain Rule Theorem 2.9, utilising the lines above when $\mathbf{w}$ and $\mathbf{b}$ appear, also recalling Proposition 2.8:

$$\begin{aligned}
\nabla_{\mathbf{w}^{(L)}} E &= (D_{\mathbf{w}^{(L)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} D_{\mathbf{w}^{(L)}} \mathbf{z}^{(L)}\right)^T \\
\nabla_{\mathbf{b}^{(L)}} E &= (D_{\mathbf{b}^{(L)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} D_{\mathbf{b}^{(L)}} \mathbf{z}^{(L)}\right)^T \\
\nabla_{\mathbf{w}^{(L-1)}} E &= (D_{\mathbf{w}^{(L-1)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} \boldsymbol{\sigma}^{(L-1)} D_{\mathbf{z}^{(L-1)}} \mathbf{z}^{(L-1)}\right)^T \\
\nabla_{\mathbf{b}^{(L-1)}} E &= (D_{\mathbf{b}^{(L-1)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} \boldsymbol{\sigma}^{(L-1)} D_{\mathbf{z}^{(L-1)}} \mathbf{z}^{(L-1)}\right)^T \\
&\vdots
\end{aligned}$$

Continuing unraveling down to layer 1, we derive the following result.

Theorem 4.1. *Gradient descent formula for a neural network with L layers.*

Let i be a number more than 0 but less than L . Given the setup above, we have that:

$$\begin{aligned}
\nabla_{\mathbf{w}^{(L)}} E &= (D_{\mathbf{w}^{(L)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} D_{\mathbf{w}^{(L)}} \mathbf{z}^{(L)}\right)^T \\
\nabla_{\mathbf{b}^{(L)}} E &= (D_{\mathbf{b}^{(L)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} D_{\mathbf{b}^{(L)}} \mathbf{z}^{(L)}\right)^T \\
\nabla_{\mathbf{w}^{(i)}} E &= (D_{\mathbf{w}^{(i)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} \left(\prod_{j=L-1}^i D_{\boldsymbol{\sigma}^{(j)}} \mathbf{z}^{(j+1)} D_{\mathbf{z}^{(j)}} \boldsymbol{\sigma}^{(j)}\right) D_{\mathbf{w}^{(i)}} \mathbf{z}^{(i)}\right)^T \\
\nabla_{\mathbf{b}^{(i)}} E &= (D_{\mathbf{b}^{(i)}} E)^T = \left(D_{\boldsymbol{\sigma}^{(L)}} E D_{\mathbf{z}^{(L)}} \boldsymbol{\sigma}^{(L)} \left(\prod_{j=L-1}^i D_{\boldsymbol{\sigma}^{(j)}} \mathbf{z}^{(j+1)} D_{\mathbf{z}^{(j)}} \boldsymbol{\sigma}^{(j)}\right) D_{\mathbf{b}^{(i)}} \mathbf{z}^{(i)}\right)^T
\end{aligned}$$

Where the terms highlighted in red denote a descending right product of terms:

$$\prod_{j=L-1}^i A^{(i)} = A^{(L-1)} A^{(L-2)} \dots A^{(i+1)} A^{(i)}$$

These can be fed into the gradient descent algorithm Theorem 2.6.

The main thing that makes the above possible to interpret as an algorithm such as in Theorem 3.4, is that the results of each Jacobian above can be written in terms of the variables $\mathbf{z}^{(i)}$ and $\mathbf{a}^{(i)}$. This is achieved by substituting the relevant vectors and matrices into Propositions 3.1, 3.3, and 3.2.⁶

For example,

$$\begin{aligned}\mathbf{z}^{(j+1)} &= W^{(j+1)} \boldsymbol{\sigma}^{(j)} \left(\mathbf{z}^{(j)} \right) + \mathbf{b}^{(j+1)} \\ D_{\boldsymbol{\sigma}^{(j)}} \mathbf{z}^{(j+1)} &= W^{(j+1)}\end{aligned}\tag{Equation 3 Proposition 3.1}$$

Hence, given a fixed $\mathbf{x} = \hat{\mathbf{x}}$ and some numbers (e.g. initially randomly chosen) weights $W^{(i)}$ and bias $\mathbf{b}^{(i)}$ for each layer, we can find how our error function E changes with respect to a specific layer's $W^{(i)}$ or $\mathbf{b}^{(i)}$ with only computing the equations in Sequence 4.0.1.

Computing the equations in Sequence 4.0.1 involves feeding numbers through each in order, this is often called *forward propagation*. Computing the gradients in Theorem 3.4 is then often called *backward propagation* due to the amount of repeated Jacobians as you go “backwards” in i values – these do not need to be recomputed each time, which saves a lot of computational expense over computing each Jacobian each time.

Due to our knowledge of gradient descent, we know that this process will find a (local) minima of E when viewed as a function of the weights and bias of each layer, $W^{(i)}$ and $\mathbf{b}^{(i)}$.

This whole process is referred to as a *neural network with L layers* (the “ L layers” can be thought of as being L sets of lines between $\mathbf{a}^{(i-1)}$ and $\mathbf{a}^{(i)}$ in our diagram).

A limitation which needs to again be addressed is with our choice of error function in Proposition 3.2. This function by definition only computes the error for one $\hat{\mathbf{y}}$. Our setup in Section 3 requires a collection of such $\hat{\mathbf{y}}_{(i)}$. One approach is to calculate one step of gradient descent for each pair $(\hat{\mathbf{x}}_{(i)}, \hat{\mathbf{y}}_{(i)})$ respectively, and take the average of the new $\mathbf{x}$ as the “actual” step. We will take another approach by incorporating the entire training dataset into our neural network algorithm via tensors.

⁶The latter two are based on our choice of $\boldsymbol{\sigma}^{(i)}$ and E ; we have chosen each $\boldsymbol{\sigma}^{(i)}$ to be the sigmoid function, and E to be the square error for consistency

5 Tensors

We would like to deal with many samples of training data $\hat{\mathbf{x}}$ simultaneously. To do so, we first need to develop a framework where we can generalise gradient descent Theorem 2.6 and the chain rule Theorem 2.9 to higher dimensional objects. One can view the motivation behind doing so in the first equation of Proposition 3.1 – we have no sensible way of interpreting the three indices without collapsing them into less indices as a result of an awkward reshaping process.

A tensor is an *arbitrary (finite) dimensional array of numbers*.⁷ We say T is a *tensor of type* (r, s) or an (r, s) *tensor* for short if it has r superscript indices and s subscript indices:

$$T^{\mu_1 \mu_2 \dots \mu_r}_{\nu_1 \nu_2 \dots \nu_s}$$

In this way, a matrix A is a tensor of type $(1, 1)$; it has two indices A^i_j . We allow r or s to be zero to represent no indices. In this way, a column vector $\mathbf{x}$ is a tensor of type $(1, 0)$; it has one upper index x^i . A row vector $\mathbf{x}^T$ will be considered a tensor of type $(0, 1)$; it has one lower index x_j . A number α (often called a scalar) is a tensor of type $(0, 0)$; it has no indices.

Due to convenience, we will ignore the range of values each index can take. We are allowing each index to take on a range of values independently. If required, we could refer to an (r, s) tensor as being of shape $(n_{\mu_1}, \dots, n_{\mu_r}, m_{\nu_1}, \dots, m_{\nu_s})$ to specify the range that each index can vary over. In this way, an $(n \times m)$ matrix is a $(1, 1)$ tensor of shape (n, m) , a column vector of dimension n is a $(1, 0)$ tensor of shape (n) . This is aligned to how the Python package NumPy defines tensors in terms of shape⁸.

It is time to return to the Einstein Summation convention which we have used sporadically. Let's consider two tensors, an (r, s) tensor T and a (p, q) tensor S . If a lower/upper index of T is denoted with the same letter as an upper/lower index of S respectively, and the range of values the two indices can take are same and equal to n_i (the corresponding shape for the respective tensors are the same for that index), then we sum over the full range of that repeated index.

$$\begin{aligned} T^{\mu_1 \dots \mu_r}_{\nu_1 \dots \nu_s} S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \sigma_q} &= T^{\mu_1 \dots \mathbf{1} \dots \mu_r}_{\nu_1 \dots \nu_s} S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \mathbf{1} \dots \sigma_q} \\ &+ T^{\mu_1 \dots \mathbf{2} \dots \mu_r}_{\nu_1 \dots \nu_s} S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \mathbf{2} \dots \sigma_q} \\ &+ \dots \\ &+ T^{\mu_1 \dots \mathbf{n}_i \dots \mu_r}_{\nu_1 \dots \nu_s} S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \mathbf{n}_i \dots \sigma_q} \end{aligned}$$

Here we can see explicitly why we required that the range of values that the summed indices can take to be equal; we do not want any leftover indices after the sum. In this way, we can consider the resulting tensor to be a tensor of type $(r + p - 1, s + q - 1)$, since we have explicitly summed over one upper and one lower index. This process is usually referred to as *contracting indices*, and it is a functionality included in the Python library NumPy via the function *einsum*.

Note that we can contract multiple indices by contracting them one at a time.

We can now interpret matrix multiplication of an $(n \times m)$ matrix A and an $(m \times k)$ matrix B as contracting a $(1, 1)$ tensor A of shape (n, m) and a $(1, 1)$ tensor B of shape (m, k) :

$$(AB)^i_j = A^i_{\mathbf{k}} B^{\mathbf{k}}_j$$

The output AB is itself a $(1, 1)$ tensor of shape (n, k) .

⁷This is a small simplification of the usual notion of tensors, but it is sufficient for our purposes.

⁸NumPy does not distinguish between upper or lower indices. One reason we have been so pedantic with horizontal spacing is so that we can more simply code the order of tensor indices by reading them left to right.

5.1 Tensor Calculus: the Tensor Jacobian and Gradient

We wish to work our way to some notion of gradient descent Theorem 2.6, but for tensors. We will approach this by generalising the Jacobian Definition 2.7, generalising the chain rule Theorem 2.9 to tensors, and taking inspiration from Proposition 2.8.

For this section, let T be an (r, s) tensor, S be a (p, q) tensor, and R be an (m, n) tensor. Moreover, let $T = T(S)$ and $S = S(R)$ be sufficiently smooth functions of the components of their respective argument tensor.

Definition 5.1. The Jacobian Tensor

Let T and S be as above. Define the *Jacobian Tensor of T* denoted $D_S T$ to be an $(r + q, s + p)$ tensor defined by the following:

$$(D_S T)^{\mu_1 \dots \mu_r \sigma_1 \dots \sigma_q}_{\nu_1 \dots \nu_s \rho_1 \dots \rho_p} = \frac{\partial T^{\mu_1 \dots \mu_r}_{\nu_1 \dots \nu_s}}{\partial S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \sigma_q}}$$

Warning: pay attention to the ordering of the indices.

Notice that when T and S are (column) vectors or scalars, we recover the Jacobian defined in Definition 2.7.

We will now consider two Jacobian tensors $D_S T$ and $D_R S$. Observe the following contraction of indices:

$$\begin{aligned} (D_S T)^{\mu_1 \dots \mu_r \sigma_1 \dots \sigma_q}_{\nu_1 \dots \nu_s \rho_1 \dots \rho_p} (D_R S)^{\rho_1 \dots \rho_p \delta_1 \dots \delta_n}_{\sigma_1 \dots \sigma_q \kappa_1 \dots \kappa_m} \\ &= \frac{\partial T^{\mu_1 \dots \mu_r}_{\nu_1 \dots \nu_s}}{\partial S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \sigma_q}} \frac{\partial S^{\rho_1 \dots \rho_p}_{\sigma_1 \dots \sigma_q}}{\partial R^{\kappa_1 \dots \kappa_m}_{\delta_1 \dots \delta_n}} \\ &= \frac{\partial T^{\mu_1 \dots \mu_r}_{\nu_1 \dots \nu_s}}{\partial R^{\kappa_1 \dots \kappa_m}_{\delta_1 \dots \delta_n}} \quad (\text{repeated application of Theorem 2.5}) \\ &= (D_R T)^{\mu_1 \dots \mu_r \delta_1 \dots \delta_n}_{\nu_1 \dots \nu_s \kappa_1 \dots \kappa_m} \end{aligned}$$

Hence we have found a way of generalising the chain rule Theorem 2.9 to tensors.

Theorem 5.2. *The Chain Rule for Tensor Jacobians*

Let T be an (r, s) tensor, S be a (p, q) tensor, and R be an (m, n) tensor. Moreover, let $T = T(S)$ and $S = S(R)$ be sufficiently smooth functions of the components of their respective argument tensor.

Define a multiplication on the Jacobian Tensor $D_T S$ of type $(r + q, s + p)$ and the Jacobian Tensor $D_S T$ of type $(p + n, q + m)$ by forming a $(r + n, s + m)$ tensor via the following:

$$(D_S T D_R S)^{\mu_1 \dots \mu_r \delta_1 \dots \delta_n}_{\nu_1 \dots \nu_s \kappa_1 \dots \kappa_m} = (D_S T)^{\mu_1 \dots \mu_r \sigma_1 \dots \sigma_q}_{\nu_1 \dots \nu_s \rho_1 \dots \rho_p} (D_R S)^{\rho_1 \dots \rho_p \delta_1 \dots \delta_n}_{\sigma_1 \dots \sigma_q \kappa_1 \dots \kappa_m}$$

Then we have the following result:

$$D_S T D_R S = D_R T$$

Inspired by Proposition 2.8, we wish to define the gradient of a function that takes in tensors. First we need to define the tensor transpose.

Definition 5.3. Let S be an (r, s) tensor, then define an (s, r) tensor denoted S^T called *the transpose of S* by the following:

$$(S^T)^{\nu_1 \dots \nu_s}_{\mu_1 \dots \mu_r} = S^{\mu_1 \dots \mu_r}_{\nu_1 \dots \nu_s}$$

Note that when S is a matrix or a vector, the transpose is equivalent to our existing notion of transposing.

Definition 5.4. Gradient of a Tensor Function

Given a $(0, 0)$ tensor f and an (r, s) tensor T such that $f = f(T)$ is a sufficiently smooth function of the components of T , define the *gradient of f with respect to T* denoted $\nabla_T f$ by the following:

$$\nabla_T f = (D_T f)^T$$

Here, the superscript T denotes transposing.

Note that when T is a (column) vector, we have that the gradient in Definition 5.4 is equivalent to the gradient in Definition 2.8.

Now we are ready to define gradient descent for a tensor function E , generalising Theorem 2.6. Note that if α is a constant, and T, S are two tensors of the same type, we can define αT to be component-wise multiplication by α , and $T + S$ to be respective component-wise addition, resulting in another tensor of the same type. This is in the same spirit with how we defined the same notions for matrices and vectors in previous sections.

Theorem 5.5. *Gradient Descent of a Tensor Function with Momentum and Wind (Noise)*

Let T be an (r, s) tensor and let $E = E(T)$ be a non-negative $(0, 0)$ tensor which is a function of T in a sufficiently smooth way. Let $\alpha, \beta \in (0, 1]$ be numbers between zero and one called the learning rate and the momentum parameter respectively. Let Γ be an (r, s) tensor of random numbers all centered around 0, and let $g : \mathbb{N} \rightarrow \mathbb{R}$; $g : n \mapsto g(n)$ be a real-valued function of the current iteration called the noise parameter. Let T_n be a sequence of (r, s) tensors defined by the iterative process:

$$T_{n+1} = T_n - \alpha \nabla_T E|_{T=T_n} - \beta(T_n - T_{n-1}) + g(n)\Gamma$$

Where the bar denotes evaluating the gradient at each point $\mathbf{x}_n$ respectively. Then, for an appropriate choice of α, β, g , we have that T_n approaches a local minimum of E as n increases.

The parameters α, β, g are referred to as hyperparameters.

The simplest way to prove the above result is by reshaping each tensor into long vectors, where it becomes equivalent to Theorem 2.6. The rest of the work is then carefully writing that reshaping map and chasing tensor types. In this way, it is also clear that the result recreates Theorem 2.6 when T is a (column) vector.

6 Neural Networks with Hidden Layers via Tensor Calculus

Now that we have a sufficient framework for tensors developed in Section 5, we can tackle the question of incorporating all of our training data at once in our formulation of neural networks with hidden layers in Section 4.

Our sequence in Sequence 4.0.1 only inputs a single $\mathbf{x}$, whereas our goal in Section 3 requires us to create a set of m input-output pairs $(\hat{\mathbf{x}}_{(j)}, \hat{\mathbf{y}}_{(j)})$.

Let's define an $(n_x \times m)$ matrix $\hat{X}$ and an $(n_y \times m)$ matrix $\hat{Y}$ by having the j th column be $\mathbf{x}_{(j)}$ and $\mathbf{y}_{(j)}$ respectively:

$$\begin{aligned}\hat{X}_j^i &= \mathbf{x}_{(j)}^i \\ \hat{Y}_j^i &= \mathbf{y}_{(j)}^i\end{aligned}$$

We store each training example as a column. Consider a matrix W . Then it follows that

$$(W\hat{X})_j^i = W_{k\mathbf{x}_{(j)}}^i$$

Hence, the j th column of $W\hat{X}$ is equal to $W\hat{\mathbf{x}}_{(j)}$.

$$W\hat{X} = \begin{bmatrix} \uparrow & & \uparrow \\ W\hat{\mathbf{x}}_{(1)} & \dots & W\hat{\mathbf{x}}_{(m)} \\ \downarrow & & \downarrow \end{bmatrix}$$

Hence, we can interpret this multiplication $W\hat{X}$ as doing an individual multiplication of $W\hat{\mathbf{x}}$ for each $\hat{\mathbf{x}}$. To incorporate our notion of bias, we must define a matrix B such that each column is m repeats of the same vector $\mathbf{b}$, i.e. $B_j^i = b^i$. Then

$$W\hat{X} + B = \begin{bmatrix} \uparrow & & \uparrow \\ W\hat{\mathbf{x}}_{(1)} + \mathbf{b} & \dots & W\hat{\mathbf{x}}_{(m)} + \mathbf{b} \\ \downarrow & & \downarrow \end{bmatrix}$$

This allows us to update our algorithm in Section 4.

To keep aligned with our goal of simplicity of computation in Section 3, we can reuse the work that led to Theorem 4.1 by inserting matrices when applying weights and then a non-linear function σ . We still call intermediate layers *hidden layers*. Let L be a number more than 0 called *the number of layers*. Define a sequence of matrices $W^{(i)}$, $Z^{(i)}$, $A^{(i)}$, (column) vectors $\mathbf{b}^{(i)}$ which we extend to matrices $B^{(i)}$ by having each column be $\mathbf{b}^{(i)}$, and *activation functions* $\sigma^{(i)}$ which take matrices to matrices of the same dimension, for $i = 1, \dots, L$ by:

$$\begin{aligned}A^{(0)} &= X \\ Z^{(i)} &= W^{(i)}A^{(i-1)} + B^{(i)} \\ A^{(i)} &= \sigma^{(i)}(Z^{(i)}) \\ Y &= A^{(L)} \\ E &= E(Y; \hat{Y})\end{aligned}\tag{6.0.1}$$

Where E is a an error function of Y using $\hat{Y}$ as a fixed goal. The components of $W^{(i)}$ and $\mathbf{b}^{(i)}$ are called the *weights and bias for layer i* respectively. Note that we are free to choose each $A^{(i)}$

to have as many rows as we would like whenever i is not 0 or L , where it must have the same number of rows as X and Y respectively. The number of columns of $A^{(i)}$ is always m , the number of samples. Moreover, given a choice of number of rows for each $A^{(i)}$ as n_i , the dimensions of each $W^{(i)}$ are determined to be $(n_i \times n_{i-1})$ respectively, and $\mathbf{b}^{(i)}$ to be a (column) vector of length n_i , making the $B^{(i)}$ into $(n_i \times m)$ matrices respectively. Note we interpret $n_0 = n_x$ and $n_L = n_y$.

We then wish to find the global minima of $E(W^{(1)}, \mathbf{b}^{(1)}, \dots, W^{(L)}, \mathbf{b}^{(L)}; \mathbf{x})$, considering $\mathbf{x}$ fixed.

Due to the results and notation we have developed in Section 5, we can follow an identical procedure to that which we conducted leading up to the main result Theorem 4.1.

Theorem 6.1. *Gradient descent formula for a neural network with L layers, using tensor calculus.*

Let i be a number more than 0 but less than L . Given the setup above, we have that:

$$\begin{aligned}\nabla_{W^{(L)}} E &= (D_{W^{(L)}} E)^T = \left(D_{\sigma^{(L)}} E D_{Z^{(L)}} \sigma^{(L)} D_{W^{(L)}} Z^{(L)} \right)^T \\ \nabla_{\mathbf{b}^{(L)}} E &= (D_{\mathbf{b}^{(L)}} E)^T = \left(D_{\sigma^{(L)}} E D_{Z^{(L)}} \sigma^{(L)} D_{\mathbf{b}^{(L)}} Z^{(L)} \right)^T \\ \nabla_{W^{(i)}} E &= (D_{W^{(i)}} E)^T = \left(D_{\sigma^{(L)}} E D_{Z^{(L)}} \sigma^{(L)} \left(\prod_{j=L-1}^i D_{\sigma^{(j)}} Z^{(j+1)} D_{Z^{(j)}} \sigma^{(j)} \right) D_{W^{(i)}} Z^{(i)} \right)^T \\ \nabla_{\mathbf{b}^{(i)}} E &= (D_{\mathbf{b}^{(i)}} E)^T = \left(D_{\sigma^{(L)}} E D_{Z^{(L)}} \sigma^{(L)} \left(\prod_{j=L-1}^i D_{\sigma^{(j)}} Z^{(j+1)} D_{Z^{(j)}} \sigma^{(j)} \right) D_{\mathbf{b}^{(i)}} Z^{(i)} \right)^T\end{aligned}$$

Where the terms highlighted in red denote a descending right tensor product of tensor jacobian terms, as defined in Section 5:

$$\prod_{j=L-1}^i D_{A^{(i-1)}} A^{(i)} = D_{A^{(L-2)}} A^{(L-1)} D_{A^{(L-3)}} A^{(L-2)} \dots D_{A^{(i)}} A^{(i+1)} D_{A^{(i-1)}} A^{(i)}$$

These can be fed into the gradient descent algorithm Theorem 2.6.

Now, all that is left is to calculate the relevant tensor Jacobians⁹ to consider the above as an algorithm.

Proposition 6.2. *Let Z , W , A , B , $\mathbf{b}$ be as in the setup above, i.e.*

$$Z = WA + B$$

Then

1. $(D_W Z)^{il}_{jk} = A^l_j \delta^i_k$
2. $(D_A Z)^{il}_{jk} = W^i_k \delta^l_j$
3. $(D_{\mathbf{b}} Z)^i_{jk} = \delta^i_k$

Note that the derivative is independent of j since each column of B is identical.

⁹This section will be done in less detail due to the bulk of the work being very similar to that in previous sections, namely Section 3 and Section 4.

Proof. Firstly write

$$Z^i_j = W^i_m A^m_j + B^i_j = W^i_m A^m_j + \mathbf{b}^i$$

Then continue:

1. $(D_W Z)^{il}_{jk}$

$$\begin{aligned} (D_W Z)^{il}_{jk} &= \frac{\partial Z^i_j}{\partial W^{k_l}} \\ &= \partial_{W^{k_l}} (W^i_m A^m_j) \\ &= A^m_j \partial_{W^{k_l}} (W^i_m) \\ &= A^m_j \delta^i_k \delta^l_m \\ &= A^l_j \delta^i_k \end{aligned}$$

2. $(D_A Z)^{il}_{jk}$

$$\begin{aligned} (D_A Z)^{il}_{jk} &= \frac{\partial Z^i_j}{\partial A^{k_l}} \\ &= \partial_{A^{k_l}} (W^i_m A^m_j) \\ &= W^i_m \partial_{A^{k_l}} (A^m_j) \\ &= W^i_m \delta^m_k \delta^l_j \\ &= W^i_k \delta^l_j \end{aligned}$$

3. $(D_{\mathbf{b}} Z)^i_{jk}$

$$\begin{aligned} (D_{\mathbf{b}} Z)^i_{jk} &= \frac{\partial Z^i_j}{\partial \mathbf{b}^k} \\ &= \partial_{\mathbf{b}^k} (\mathbf{b}^i) \\ &= \delta^i_k \end{aligned}$$

□

Proposition 6.3. Let Z be as above, and let σ be the component-wise extension of a one-variable activation function σ , i.e.

$$(\sigma(Z))^i_j = \sigma(Z^i_j)$$

Then

$$(D_Z \sigma)^{il}_{jk} = \delta^i_k \delta^l_j \sigma'$$

Proof. We plug in at a point Z^i_j denoted with a vertical bar and use the chain rule:

$$\begin{aligned} (D_Z \sigma)^{il}_{jk} \Big|_{Z^i_j} &= \frac{\partial \sigma}{\partial Z^{k_l}} \Big|_{Z^i_j} \\ &= \frac{\partial \sigma(Z^i_j)}{\partial Z^{k_l}} \\ &= \sigma' \Big|_{Z^i_j} \partial_{Z^{k_l}} Z^i_j \\ &= \delta^i_k \delta^l_j \sigma' \Big|_{Z^i_j} \end{aligned}$$

This holds for all Z^i_j and thus completes the proof.

□

In the case of the sigmoid function, we can reuse our work in Proposition 3.3 to find the following.

Corollary 6.4. *If σ is the component-wise extension of the sigmoid function, then*

$$(D_Z \sigma)^{il}_{jk} \Big|_{Z^{i_j}} = \delta^i_j \delta^l_j (\sigma(Z^i_j) (1 - \sigma(Z^i_j)))$$

□

Other activation functions are commonly used, such as the softmax function, ReLU, tanh, or even just the identity function (i.e. do nothing). These each have different purposes and strengths, and the interested reader should continue by looking into the differences.

The process above for the sigmoid function can then be repeated on e.g. the column-wise or component-wise extension of these functions to compute the tensor Jacobian.

A sketch is given for the softmax function below.¹⁰

Example. Let σ_s be the column-wise extension of the softmax function σ_s to matrices. I.e. if

$$(\sigma_s(\mathbf{z}))^i = \frac{e^{\mathbf{z}^i}}{\sum_k e^{\mathbf{z}^k}}$$

And considering Z as a sequence of column vectors $Z^i_j = \mathbf{z}_{(j)}^i$,

$$\sigma_s(Z)^i_j = (\sigma_s(\mathbf{z}_{(j)}))^i$$

Then

$$(D_Z \sigma)^{il}_{jk} \Big|_{Z^{i_j}} = \delta^l_j \sigma_s(Z)^i_l (\delta^i_k - \sigma_s(Z)^k_l)$$

Proof. (sketch)

Using a standard result that

$$\frac{\partial \sigma_s(\mathbf{z})^i}{\partial \mathbf{z}^j} = \sigma_s(\mathbf{z})^i (\delta^i_j - \sigma_s(\mathbf{z})^j)$$

Where we are being a bit looser with our indices and not summing above, we compute:

$$\begin{aligned} (D_Z \sigma)^{il}_{jk} \Big|_{Z^{i_j}} &= \frac{\partial \sigma_s^i_j}{\partial Z^k_l} \Big|_{Z^{i_j}} \\ &= \frac{\partial \sigma_s^i_j}{\partial \mathbf{z}_{(l)}^k} \Big|_{\mathbf{z}_{(j)}^i} \\ &= \delta^l_j \sigma_s(\mathbf{z}_{(l)})^i (\delta^i_k - \sigma_s(\mathbf{z}_{(l)})^k) \\ &= \delta^l_j \sigma_s(Z)^i_l (\delta^i_k - \sigma_s(Z)^k_l) \end{aligned}$$

□

We now continue with our favourite E , the average mean squared error.

Proposition 6.5. *Let E be the average mean squared error for m samples, and $\|\cdot\|$ denote the Euclidean norm so that we define E as:*

$$E(Y; \hat{Y}) = \frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_{(i)} - \hat{\mathbf{y}}_{(i)}\|^2$$

¹⁰Here we start to use explicit summing notation using $\sum$ where it would be too cumbersome to use Einstein summation convention.

Then

$$(D_Y E)^l{}_k = \frac{1}{m} \left(Y - \hat{Y} \right)^T$$

Where we, as usual, view Y and $\hat{Y}$ as a sequence of m column vectors $\mathbf{y}_{(i)}$ and $\hat{\mathbf{y}}_{(i)}$, and we are using superscript T to denote transpose.

Proof. We compute, being careful with indices for to keep track of transpose,

$$\begin{aligned} (D_Y E)^l{}_k &= \frac{\partial E}{\partial Y^k{}_l} \\ &= \frac{\partial E}{\partial \mathbf{y}_{(l)}^k} \\ &= \frac{1}{2m} \sum_{i=1}^m \partial_{\mathbf{y}_{(l)}^k} \left(\|\mathbf{y}_{(i)} - \hat{\mathbf{y}}_{(i)}\|^2 \right) \\ &= \frac{1}{2m} \sum_{i=1}^m \sum_{j=1}^{n_y} \partial_{\mathbf{y}_{(l)}^k} \left(\mathbf{y}_{(i)}^j - \hat{\mathbf{y}}_{(i)}^j \right)^2 \\ &= \frac{1}{2m} 2\delta^i{}_l \delta^k{}_j \left(\mathbf{y}_{(i)}^j - \hat{\mathbf{y}}_{(i)}^j \right) \\ &= \frac{1}{m} \left(\mathbf{y}_{(l)}^k - \hat{\mathbf{y}}_{(l)}^k \right) \\ &= \frac{1}{m} \left(Y - \hat{Y} \right)^T \end{aligned}$$

□

And finally, we can plug in the results from Proposition 6.2, Proposition 6.5, and Corollary 6.4¹¹ into the gradient descent formula for a neural network with L layers using tensor calculus Theorem 6.1 to compute the algorithm.

¹¹The latter two are only in the case of using the sigmoid activation function and the average mean square error for our error function.

7 Implementation

The implementation can be found at https://github.com/Jaffulee/deep_neural_network

This algorithm was implemented in Python using NumPy directly, making heavy use of tensors and the function `einsum`. This is useful since all of the results here were derived making heavy use of Einstein Summation convention, and thus can be implemented directly.

Due to this being a mainly theoretic-centric demonstration for learning purposes, the efficiency of the algorithm is not a focus. Many further optimisations are made in packages like **PyTorch** or **TensorFlow**. For example, many of the tensor multiplications can be simplified further analytically before coding, whereas the implementation here directly computes the tensor multiplications as in Theorem 6.1.

Further details can be found in the repository linked above.